

SIMATS ENGINEERING

CO4 AT3 Data Interpretation

NAME : Hitesh C

REG NO : 192524005

COURSE CODE : CSA1401

COURSE NAME : Compiler Design

COURSE FACULTY : AJITHA V

Q1. Given the three-address code:

$t1 = a * b$

$t2 = t1 + c$

$t3 = t2 - d$

Tasks:

- (i) Analyze the sequence of computations performed by the intermediate code.**
- (ii) Construct the equivalent expression tree for the given code.**
- (iii) Explain how temporary variables ($t1$, $t2$) help reduce recomputation and improve execution efficiency.**

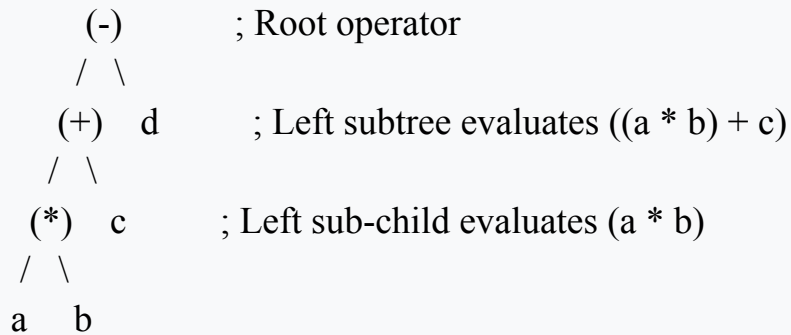
Answer:

(i) Computation Sequence Analysis:

- Statement 1 ($t1 = a * b$): Multiplies variables a and b , storing the product in temporary variable $t1$.
- Statement 2 ($t2 = t1 + c$): Adds variable c to intermediate result $t1$, storing the sum in temporary variable $t2$.
- Statement 3 ($t3 = t2 - d$): Subtracts variable d from intermediate sum $t2$, yielding final result $t3$.
- Order of Operations: Respects arithmetic precedence where multiplication ($*$) is evaluated prior to addition ($+$) and subtraction ($-$).

(ii) Equivalent Expression Tree:

Infix Expression: $((a * b) + c) - d$



(iii) Efficiency Benefits of Temporary Variables (t1, t2):

1. **Expression Linearization:** Breaks down complex nested expressions into simple 3-address binary instructions matching hardware machine ISAs.
2. **Register Preservation:** Temporaries store intermediate sub-expression values directly in fast CPU registers, eliminating redundant memory writes and reads.
3. **Optimization Support:** Enables compiler optimization passes (such as common subexpression elimination and instruction scheduling) to track data flow across instructions.

Q2. Given the symbol table entries:

Variable	Type	Size
p	int	4
q	float	8
r	char	1
s	double	8

Tasks:

- (i) Interpret how memory is allocated for each variable.**
- (ii) Calculate the total memory requirement for the variables.**
- (iii) Explain the effect of memory alignment and padding on the overall storage size.**

Answer:

(i) Memory Allocation Interpretation:

- Variable 'p': Allocated 4 contiguous bytes in memory for standard 32-bit integer storage.
- Variable 'q': Allocated 8 contiguous bytes in memory for double-precision float representation.
- Variable 'r': Allocated 1 byte in memory for ASCII character storage.
- Variable 's': Allocated 8 contiguous bytes in memory for 64-bit IEEE 754 double storage.

(ii) Total Raw Memory Calculation:

Total Raw Memory = Size(p) + Size(q) + Size(r) + Size(s)

Total Raw Memory = 4 + 8 + 1 + 8 = 21 bytes.

(iii) Memory Alignment & Padding Effects:

Modern hardware architectures mandate alignment where n-byte data types reside on memory addresses divisible by n bytes:

Offset 0..3 : Variable p (int, 4 bytes)

Offset 4..7 : Padding (4 bytes) -> Aligns 8-byte variable q to address 8

Offset 8..15 : Variable q (float, 8 bytes)

Offset 16 : Variable r (char, 1 byte)

Offset 17..23 : Padding (7 bytes) -> Aligns 8-byte variable s to address 24

Offset 24..31 : Variable s (double, 8 bytes)

Total Memory with Padding = 4 + 4(pad) + 8 + 1 + 7(pad) + 8 = 32 bytes.

Effect of Padding: Padding increases total storage from 21 bytes to 32 bytes, trading 11 bytes of memory overhead to ensure single-cycle alignment reads across the CPU data bus.

Q3. Given the intermediate code:

t1 = x + y

t2 = t1 * z

t3 = x + y

t4 = t3 - w

Tasks:

- (i) Trace the computation step-by-step.
- (ii) Identify opportunities for common subexpression elimination.
- (iii) Rewrite the optimized intermediate code after removing redundant computations.

Answer:

(i) Step-by-Step Computation Trace:

- Step 1 ($t1 = x + y$): Calculates sum of operands x and y , assigning result to $t1$.
- Step 2 ($t2 = t1 * z$): Multiplies sum $t1$ by z , assigning result to $t2$.
- Step 3 ($t3 = x + y$): Re-calculates identical sum of operands x and y , assigning result to $t3$.
- Step 4 ($t4 = t3 - w$): Subtracts w from $t3$, assigning result to $t4$.

(ii) Common Subexpression Identification:

- Subexpression Analysis: Expression $(x + y)$ is computed twice: at statement 1 into $t1$ and at statement 3 into $t3$.
- Validity Check: Neither variable x nor variable y is modified between statement 1 and statement 3.
- Conclusion: Statement 3 ($t3 = x + y$) is a redundant Common Subexpression.

(iii) Rewritten Optimized Intermediate Code:

```
t1 = x + y    ; Compute (x + y) once
t2 = t1 * z   ; Use t1 in multiplication
t4 = t1 - w   ; Reuse t1 directly, eliminating redundant t3 computation
```

Optimization Impact: Eliminates 1 addition operation and 1 temporary variable allocation ($t3$), reducing execution time and code size.

Q4. Given the condition:

if ($a < b \parallel c > d$)

Intermediate code:

```
if a < b goto L1
if c > d goto L1
goto L2
L1: ...
```

Tasks:

- (i) Interpret the control flow represented by the intermediate code.**
- (ii) Explain how short-circuit evaluation is implemented.**
- (iii) Illustrate the execution flow for different values of a, b, c, and d.**

Answer:**(i) Control Flow Interpretation:**

- First Test (if $a < b$ goto L1): Checks if $a < b$. If true, branches immediately to L1 (True Branch).
- Second Test (if $c > d$ goto L1): If first test is false, checks if $c > d$. If true, branches to L1.
- Fall-Through (goto L2): If both tests evaluate to false, execution branches to L2 (False Branch).
- Label L1: Entry point for statement execution when condition evaluates to TRUE.

(ii) Implementation of Short-Circuit Evaluation:

Logical OR ($\parallel$) short-circuits on TRUE: As soon as the left operand ($a < b$) evaluates to true, the compiler skips evaluating the right operand ($c > d$) and jumps directly to L1. The second condition is tested only when the first condition fails.

(iii) Execution Flow Scenarios for Sample Input Values:

1. Scenario 1 ($a = 2, b = 5, c = 10, d = 20$) -> $a < b$ is TRUE:
 - 'if $2 < 5$ goto L1' evaluates TRUE -> Execution jumps directly to L1. ' $c > d$ ' is NEVER evaluated.
2. Scenario 2 ($a = 7, b = 5, c = 30, d = 20$) -> $a < b$ is FALSE, $c > d$ is TRUE:
 - 'if $7 < 5$ goto L1' evaluates FALSE -> Falls through to second test.
 - 'if $30 > 20$ goto L1' evaluates TRUE -> Execution jumps to L1.
3. Scenario 3 ($a = 7, b = 5, c = 10, d = 20$) -> Both conditions FALSE:
 - 'if $7 < 5$ goto L1' evaluates FALSE -> Falls through.
 - 'if $10 > 20$ goto L1' evaluates FALSE -> Falls through to 'goto L2'. Execution branches to L2.

Q5. Given:**Instruction Code**

120 if p > q goto _
121 goto _

Backpatch lists:**truelist = {120}****falselist = {121}****Tasks:**

- (i) Analyze how backpatching is applied to the given instructions.
- (ii) Fill in the appropriate target labels for the pending jumps.
- (iii) Explain how control flow is resolved using the truelist and falselist.

Answer:**(i) Analysis of Backpatching Application:**

During single-pass syntax-directed translation, forward jump instructions (quads 120 and 121) are generated before their target labels are known. The compiler leaves destination targets blank ('goto _') and collects quad indices into lists: trueList = {120} and falseList = {121}.

(ii) Binding Pending Jumps to Target Labels:

Let L_TRUE (e.g., Quad 122) be the destination for condition TRUE, and L_FALSE (e.g., Quad 130) be the destination for condition FALSE:

- backpatch(trueList, L_TRUE) -> Updates instruction 120
- backpatch(falseList, L_FALSE) -> Updates instruction 121

120: if p > q goto L_TRUE ; Resolved via backpatch(trueList, L_TRUE)
121: goto L_FALSE ; Resolved via backpatch(falseList, L_FALSE)

(iii) Control Flow Resolution Explanation:

Backpatching resolves control flow dynamically without needing a multi-pass AST traversal. By maintaining lists of instruction addresses, the compiler traverses the target list when a basic block label is reached, filling in blank fields in-place.

Q6. Given the expression:

result = a * b + c / d

Tasks:

- (i) Generate the corresponding three-address code.**
- (ii) Explain the order of evaluation based on operator precedence.**
- (iii) Describe the role of temporary variables in evaluating the expression.**

Answer:

(i) Three-Address Code Generation:

```
t1 = a * b    ; Evaluate multiplication (a * b)
t2 = c / d    ; Evaluate division (c / d)
t3 = t1 + t2   ; Add intermediate results (t1 + t2)
result = t3    ; Store final result in target variable
```

(ii) Order of Evaluation Based on Operator Precedence:

- Precedence Rules: Multiplication (*) and division (/) have higher precedence than addition (+). Both * and / associate left-to-right.
- Step 1: Sub-expression (a * b) is evaluated first into t1.
- Step 2: Sub-expression (c / d) is evaluated second into t2.
- Step 3: Addition (t1 + t2) is performed third into t3.
- Step 4: Final value is assigned to result.

(iii) Role of Temporary Variables (t1, t2, t3):

1. Operand Limit Enforcement: Enforces standard 3-address format (at most 2 source operands, 1 operator per instruction).
2. Precedence Boundary Preservation: Temporaries hold sub-expression outputs safely until higher-precedence terms finish computing.
3. Hardware Register Mapping: Direct mapping to machine registers facilitates optimal register allocation by target code generators.

Q7. Given:

```
switch(choice)
{
    case 1: OPEN;
```

```
case 2: SAVE;  
case 3: EXIT;  
}
```

Tasks:

- (i) Translate the switch statement into intermediate code using conditional jumps.**
- (ii) Explain the execution of conditional branches for each case.**
- (iii) Describe how a jump table can optimize the execution of the switch statement.**

Answer:

(i) Intermediate Code using Conditional Jumps:

```
100: if choice == 1 goto L_CASE1 ; Test case 1  
101: if choice == 2 goto L_CASE2 ; Test case 2  
102: if choice == 3 goto L_CASE3 ; Test case 3  
103: goto L_END ; Fall-through/Default exit  
104: L_CASE1: OPEN ; Execute OPEN action  
goto L_END  
105: L_CASE2: SAVE ; Execute SAVE action  
goto L_END  
106: L_CASE3: EXIT ; Execute EXIT action  
goto L_END  
107: L_END:
```

(ii) Execution of Conditional Branches for Each Case:

- Case 1 (choice = 1): Quad 100 evaluates TRUE -> Jumps to L_CASE1, executes OPEN, branches to L_END.
- Case 2 (choice = 2): Quad 100 evaluates FALSE -> Quad 101 evaluates TRUE -> Jumps to L_CASE2, executes SAVE, branches to L_END.
- Case 3 (choice = 3): Quads 100-101 evaluate FALSE -> Quad 102 evaluates TRUE -> Jumps to L_CASE3, executes EXIT, branches to L_END.
- Unmatched Choice: Quads 100-102 evaluate FALSE -> Quad 103 executes 'goto L_END', bypassing all case handlers.

(iii) Jump Table Optimization Description:

A jump table stores target code addresses in an array indexed by (choice - 1):

TABLE = [L_CASE1, L_CASE2, L_CASE3]

Instead of checking sequential if-conditions ($O(N)$ time complexity), the compiler generates a bounds check followed by an indirect jump: goto TABLE[(choice - 1) * 4]. This guarantees $O(1)$ constant-time branching.

Q8. Given the function call:

total = compute(x, y)

Intermediate representation:

param x

param y

call compute, 2

Tasks:

- (i) Explain the parameter passing mechanism used in the procedure call.
- (ii) Describe the structure and purpose of the activation record.
- (iii) Explain how the return value is transferred back to the calling function.

Answer:

(i) Parameter Passing Mechanism:

1. 'param x' and 'param y' evaluate arguments x and y and push them onto the execution stack (or load them into calling convention registers like r0, r1).
2. 'call compute, 2' pushes the return address and saved frame pointer onto the stack, transferring execution control to procedure compute with parameter count 2.

(ii) Structure & Purpose of Activation Record (Stack Frame):

+-----+	
Actual Parameters (x, y)	-> Passed by caller (FP + 8, FP + 12)
+-----+	
Return Address	-> Pushed by hardware CALL (FP + 4)
+-----+	
Control Link (Saved Frame Pointer FP)	-> Saved caller frame boundary (FP + 0)

+-----+
Saved Registers & Local Variables -> Allocated for callee execution
+-----+

Purpose: Manages dynamic state, local scope lifetimes, control recursion, and parameter access for each active procedure invocation.

(iii) Return Value Transfer Mechanism:

When function compute finishes execution, it stores its return value in hardware return register EAX (or R0). Upon executing the return instruction, control reverts to the caller, which emits machine code copying EAX into variable total ('total = EAX').

Q9. Given:

if (m > n && p < q)

Intermediate code:

if m > n goto L1
goto L3

L1: if p < q goto L2
goto L3

Tasks:

- (i) Interpret the control flow represented by the code.**
- (ii) Explain how true and false lists are maintained during code generation.**
- (iii) Describe how backpatching resolves unresolved jump targets.**

Answer:

(i) Control Flow Interpretation:

- First Check (if m > n goto L1): Tests m > n. If true, jumps to L1 to evaluate the second condition.
- Short-Circuit False (goto L3): If m > n is false, short-circuits the AND condition and jumps directly to L3 (False Branch).

- Second Check (L1: if $p < q$ goto L2): Tests $p < q$. If true, jumps to L2 (True Branch).
- Second Check False (goto L3): If $p < q$ is false, branches to L3 (False Branch).

(ii) Maintenance of True and False Lists:

- For $B1 = (m > n)$: truelist = [100], falselist = [101].
- For $B2 = (p < q)$: truelist = [102], falselist = [103].
- In Logical AND ($B1 \ \&\& \ B2$):
 1. $B1$.truelist is backpatched to M.quad (label L1 at Quad 102).
 2. Overall B.truelist becomes $B2$.truelist = [102] (targeting L2).
 3. Overall B.falselist becomes $\text{merge}(B1.\text{falselist}, B2.\text{falselist}) = [101, 103]$ (targeting L3).

(iii) Backpatching Resolution Explanation:

Backpatching resolves jump targets in a single syntax pass by maintaining linked lists of un-filled instruction locations. When target labels (L1, L2, L3) are synthesized, $\text{backpatch}(\text{list}, \text{label})$ traverses the target list and updates jump targets in-place.

Q10. Given the intermediate code:

```
t1 = p * q
t2 = t1 + r
t3 = t2 * s
```

Tasks:

- Analyze the data dependencies among the statements.
- Explain how computed values are reused through temporary variables.
- Construct the corresponding data-flow representation and identify possible optimization opportunities.

Answer:

(i) Data Dependency Analysis:

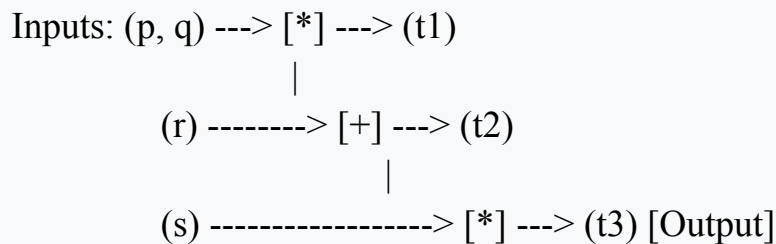
- Statement 1 ($t1 = p * q$): Computes t1. Dependencies: p, q.
- Statement 2 ($t2 = t1 + r$): Computes t2. Has a True Data Dependency (Read-After-Write / RAW) on t1 produced in Statement 1.

- Statement 3 ($t3 = t2 * s$): Computes $t3$. Has a True Data Dependency (RAW) on $t2$ produced in Statement 2.
- Dependence Chain: $S1 \rightarrow S2 \rightarrow S3$ (Strict sequential data dependence chain).

(ii) Value Reuse via Temporary Variables:

- Temporary $t1$ stores the product of p and q , passing it directly into Statement 2 without recomputing $p * q$.
- Temporary $t2$ stores the intermediate sum ($t1 + r$), passing it directly into Statement 3 without recomputing ($t1 + r$).
- Eliminates redundant computations and minimizes memory access by retaining intermediate values in CPU registers.

(iii) Data-Flow Representation & Optimization Opportunities:



Optimization Opportunities:

1. Multiply-Accumulate (MAC) Fusion: If target hardware supports MAC instructions, Statement 1 and Statement 2 can be fused into a single MAC instruction: $t2 = \text{MAC}(p, q, r)$.
2. Register Allocation Optimization: Because $t1$ is consumed immediately in $S2$ and $t2$ is consumed immediately in $S3$, a compiler register allocator can reuse a single temporary register for $t1$, $t2$, and $t3$, reducing register pressure.